Лабораторная работа №4. Dockerfile. Создание собственных образов

Тема: Создание Dockerfile, сборка образа и запуск Express API

Цель: Научиться создавать собственный Docker-образ Node.js/Express, управлять контекстом сборки, проверять работу контейнера и диагностировать ошибки.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №4. Dockerfile. Создание собственных образов. Продолжительность — 4 академических часа (180 минут). Работа выполняется индивидуально в Debian 13 с установленным Docker Engine. Цель — создать минимальный REST API на Express.js, написать Dockerfile, собрать собственный образ и проверить работу контейнера.

2. Планируемые результаты

1. Описывать назначение Dockerfile и контекста сборки.
2. Создавать файлы приложения и Dockerfile.
3. Выполнять docker build и docker run.
4. Проверять HTTP-ответ, состояние контейнера и логи.
5. Понимать назначение .dockerignore и кэша сборки.
6. Находить и устранять типичные ошибки сборки и запуска.

3. Оборудование и исходные условия

7. Debian 13 с Docker Engine; доступ к сети для скачивания базового образа и npm-пакетов.
8. Терминал с sudo и текстовый редактор (nano или VS Code).
9. curl для проверки HTTP (если не установлен, установить отдельно).
10. Node.js/npm на хосте не обязательны: npm-команды подготовки зависимостей можно выполнить во временном контейнере.
11. Пример предполагает свободный порт 3000 на Debian.

4. Схема и структура проекта

Схема запроса

```
Клиент (curl) -> Debian:3000 -> Docker-контейнер:3000 -> Express GET /api/health
```

Файлы проекта

```
express-api/
├── app.js
├── package.json
├── package-lock.json
├── Dockerfile
└── .dockerignore
```

Файлы располагаются в одном каталоге. В Dockerfile используется рабочая директория /app. База данных на данном этапе не требуется.

5. Подготовка учебной среды

Этап 1. Убедитесь, что Docker Engine работает. Команда обращается к Docker Daemon.

```
Проверка Docker

sudo docker version
sudo docker info
```

Критерий продолжения: в выводе docker version присутствуют Client и Server, а docker info завершается без ошибки подключения к демону.

Этап 2. Проверьте наличие curl; если утилита отсутствует, установите её.

```
Проверка curl

command -v curl
```

Если команда не выводит путь к программе, выполните установку:

```
Установка curl при необходимости

sudo apt update
sudo apt install -y curl
```

Проверка: curl --version выводит установленную версию.

```
Подтверждение установки

curl --version
```

Этап 3. Создайте каталог проекта и перейдите в него.

```
Создание проекта

mkdir -p ~/express-api
cd ~/express-api
pwd
```

Ожидается путь, заканчивающийся на /express-api. Дальнейшие команды выполняйте из этого каталога.

6. Создание минимального приложения

Этап 4. Создайте app.js. Пример использует ES Modules и слушает 0.0.0.0, чтобы HTTP-запросы могли поступать через опубликованный порт контейнера.

```
Содержимое app.js

import express from "express";

const app = express();

app.get("/api/health", (req, res) => {
  res.json({ status: "ok" });
```

```
});

app.listen(3000, "0.0.0.0", () => {
  console.log("API listening on port 3000");
});
```

Создайте файл app.js через текстовый редактор и вставьте код. Затем проверьте, что он существует.

Проверка файла

```
ls -l app.js
```

Этап 5. Создайте package.json с типом module и зависимостью express. Для воспроизводимой установки нужен package-lock.json, поэтому следующим шагом сформируем lock-файл через npm в отдельном временном контейнере.

Содержимое package.json

```
{
  "name": "express-api-docker-lab",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Создайте файл package.json. В нём указана зависимость Express 5; точные устанавливаемые версии будут зафиксированы npm в package-lock.json. В рамках занятия для корректной работы необходимо наличие версии Express, удовлетворяющей указанному диапазону.

Этап 6. Сформируйте package-lock.json. Используем временный контейнер node:22-alpine. Текущий каталог монтируется в /app, поэтому созданный lock-файл появится в каталоге проекта на Debian. Команда npm install --package-lock-only не создаёт node_modules.

Генерация package-lock.json

```
sudo docker run --rm -v "$PWD":/app -w /app node:22-alpine npm install --package-lock-only
```

Ожидаемый результат: появится package-lock.json. Если Docker не может скачать образ или npm-пакеты, проверьте подключение к сети.

Проверка файлов зависимостей

```
ls -l package.json package-lock.json
```

7. Создание Dockerfile

Этап 7. Создайте текстовый файл с именем Dockerfile без расширения.

Содержимое Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Значение инструкций

Инструкция	Объяснение
FROM	Выбор образа Node.js в Alpine Linux.
WORKDIR	Рабочая директория /app внутри образа.
COPY package*.json	Копирование package.json и package-lock.json.
RUN npm ci --omit=dev	Установка производственных зависимостей при сборке.
COPY . .	Копирование исходного кода в образ.
EXPOSE 3000	Документирование порта приложения, без публикации.
CMD	Команда запуска основного процесса контейнера.

Важно: порядок инструкций позволяет использовать кэш npm-зависимостей при изменении только app.js.

8. Создание .dockerignore

Этап 8. Создайте файл .dockerignore. Он исключит ненужные локальные файлы из контекста сборки.

Содержимое .dockerignore

```
node_modules
npm-debug.log
.git
.env
coverage
```

Проверьте структуру проекта и наличие скрытого файла .dockerignore.

Проверка структуры

```
ls -la
```

Критерий продолжения: Dockerfile, .dockerignore, app.js, package.json и package-lock.json находятся в одном каталоге.

9. Сборка Docker-образа

Этап 9. Соберите образ. Точка в конце указывает текущий каталог в качестве контекста сборки, а -t задаёт имя и тег.

```
Сборка образа

sudo docker build -t express-api:v1 .
```

Критерий продолжения: сборка завершилась без ошибок. После успешной сборки проверьте локальные образы.

```
Проверка локального образа

sudo docker image ls express-api
```

В выводе должен присутствовать REPOSITORY express-api и TAG v1.

Этап 10. Исследуйте метаданные и историю образа.

```
Метаданные и история сборки

sudo docker image inspect express-api:v1
sudo docker image history express-api:v1
```

В истории найдите операции копирования файлов и установки зависимостей. Не все инструкции формируют отдельный файловый слой; часть записывает только метаданные.

10. Запуск и тестирование

Этап 11. Создайте и запустите контейнер из собственного образа.

```
Запуск контейнера

sudo docker run -d --name api1 -p 3000:3000 express-api:v1
```

Параметр -d запускает в фоне, --name задаёт имя api1, а -p публикует порт 3000 контейнера на порту 3000 Debian.

```
Проверка контейнера

sudo docker ps
```

Критерий продолжения: api1 имеет статус Up, а в PORTS указана публикация порта 3000.

Этап 12. Запросите endpoint REST API.

```
Проверка HTTP

curl -i http://localhost:3000/api/health
```

```

```

При успешном запуске ожидается статус 200 OK и тело {"status":"ok"}. Если проверяете из Windows, а Docker работает в Debian VM, используйте IP виртуальной машины вместо localhost и убедитесь в сетевой доступности.

Этап 13. Просмотрите журналы контейнера и сведения о его состоянии.

```
Диагностика запущенного API

sudo docker logs api1
sudo docker inspect api1
```

В логах ожидается сообщение API listening on port 3000. В inspect найдите состояние Running, имя и PortBindings.

11. Диагностические эксперименты

Эксперимент А. Конфликт имени контейнера

Повторите запуск с тем же именем api1 без удаления существующего контейнера. Команда должна завершиться ошибкой конфликта имени.

```
Создание конфликта имени

sudo docker run -d --name api1 -p 3001:3000 express-api:v1
```

Объясните: имена контейнеров в одном Docker Daemon должны быть уникальными. Восстановление: повторно запустите новый контейнер с другим именем и удалите его после проверки.

```
Восстановление после конфликта имени

sudo docker run -d --name api2 -p 3001:3000 express-api:v1
curl -i http://localhost:3001/api/health
sudo docker stop api2
sudo docker rm api2
```

Критерий: временный контейнер api2 удалён, а api1 по-прежнему работает.

Эксперимент Б. Конфликт опубликованного порта

Попробуйте запустить контейнер с уникальным именем, но тем же портом хоста 3000. Порт уже занят контейнером api1; ожидается ошибка публикации порта.

```
Создание конфликта порта

sudo docker run -d --name api-port-test -p 3000:3000 express-api:v1
```

Docker может оставить контейнер в состоянии Created после неуспешного запуска. Проверьте наличие объекта и удалите его безопасно.

```
Диагностика после ошибки
```

```
sudo docker ps -a
```

Удаление неуспешного контейнера

```
sudo docker rm -f api-port-test
```

Повторите запуск с незанятым портом хоста 3002 и проверьте HTTP-ответ.

Исправление порта и очистка

```
sudo docker run -d --name api-port-test -p 3002:3000 express-api:v1
curl -i http://localhost:3002/api/health
sudo docker stop api-port-test
sudo docker rm api-port-test
```

Критерий: запрос на порт 3002 успешен, контейнер api-port-test удалён.

Эксперимент B. Кэш сборки

Измените строку status в app.js с ok на ready. Затем повторно соберите образ под новым тегом.

Повторная сборка

```
sudo docker build --progress=plain -t express-api:v2 .
```

Изучите журнал сборки. При неизменных package.json и package-lock.json сборщик обычно использует кэш для шага RUN npm ci; поведение кэша зависит от состояния сборщика и предыдущих сборок.

Запустите новую версию на отдельном порту 3003, проверьте изменённый ответ и удалите тестовый контейнер.

Проверка версии v2

```
sudo docker run -d --name api-v2 -p 3003:3000 express-api:v2
curl -i http://localhost:3003/api/health
sudo docker stop api-v2
sudo docker rm api-v2
```

Ожидаемый результат: HTTP 200 и JSON со status ready. В отчёте сравните v1 и v2.

12. Самостоятельное задание

Без копирования готового Dockerfile создайте собственный вариант образа express-api:student. Измените приложение: добавьте обработчик GET /api/info, возвращающий JSON с названием учебной работы и версией приложения. Используйте тот же порт 3000 внутри контейнера и опубликуйте его на порту 3004 Debian.

12. Дополните app.js маршрутом /api/info.
13. Соберите образ с тегом express-api:student.
14. Запустите контейнер с именем student-api и портами 3004:3000.
15. Проверьте оба маршрута /api/health и /api/info через curl.

16. Покажите вывод docker ps и docker logs student-api.
17. Остановите и удалите контейнер, сохранив образ.

Самостоятельно выберите содержимое JSON /api/info, но включите минимум два поля: lab и version. Проверка считается успешной при HTTP 200 и корректном JSON.

13. Итоговая проверка и уборка

```
Итоговый список объектов

sudo docker ps -a
sudo docker image ls express-api
```

Остановите и удалите только контейнеры этой лабораторной работы. Не выполняйте глобальные команды docker system prune и docker volume prune.

```
Очистка учебных контейнеров

sudo docker stop api1 student-api
sudo docker rm api1 student-api
```

При успешном выполнении образы express-api:v1, express-api:v2 и express-api:student остаются локально. Если самостоятельное задание ещё не завершено, не выполняйте очистку до защиты.

14. Что должно получиться

18. Рабочий Dockerfile для Node.js/Express.
19. Собственные локальные образы express-api:v1, express-api:v2 и express-api:student.
20. Подтверждённый JSON-ответ API на опубликованном порту.
21. Доказательства анализа логов, inspect и истории образа.
22. Успешно исправленные диагностические ошибки с именем и портом.

15. Содержание отчёта

23. Тема, цель, ПО и окружение.
24. Схема взаимодействия клиента и контейнера.
25. Структура проекта, исходный код app.js и Dockerfile.
26. Содержимое .dockerignore.
27. Команды сборки и вывод docker image ls.
28. Команды запуска, вывод docker ps и результат curl.
29. Результаты двух диагностических экспериментов и исправления.
30. Наблюдения о кэше сборки для v2.
31. Скриншоты работы /api/info в индивидуальном задании.
32. Краткий вывод о различии образа и контейнера.

16. Контрольные вопросы

33. 1. Для чего предназначен Dockerfile?
34. 2. Что означает точка в конце команды docker build -t express-api:v1 .?
35. 3. Чем инструкция RUN отличается от CMD?
36. 4. Почему EXPOSE 3000 не открывает порт на хосте?
37. 5. Для чего нужен package-lock.json при npm ci?
38. 6. Зачем отдельно копировать package*.json до COPY . .?
39. 7. Какую роль выполняет .dockerignore?
40. 8. Почему Node.js должен слушать 0.0.0.0 в контейнере?

41. 9. Как различить ошибку сборки и ошибку запуска контейнера?
42. 10. Как проверить, что веб-приложение действительно отвечает по HTTP?
43. 11. Почему нельзя запустить два контейнера с одним именем?
44. 12. Как запустить два контейнера, слушающих одинаковый внутренний порт 3000?

17. Критерии оценивания

Оценивание

Критерий	Баллы
Подготовка структуры приложения и lock-файла	15
Корректный Dockerfile и .dockerignore	20
Сборка и анализ собственного образа	15
Запуск и проверка Express API	15
Диагностические эксперименты и восстановление	15
Самостоятельное задание /api/info	10
Отчёт и контрольные вопросы	10
Итого	100

Справочные источники: https://docs.docker.com/reference/dockerfile/ ;
https://docs.docker.com/build/building/best-practices/ ;
https://docs.docker.com/reference/cli/docker/buildx/build/ .